a complete, step-by-step study guide starting from scratch on a Windows PC — including creating a project, Python environment, installing packages, and using TrustGuard with Pandas. I'll structure it so a beginner can follow it from zero.

Lecture 3 : Coding

by : Dr. Mohammed Khalaf

Complete Step-by-Step Guide: TrustGuard Validation on Windows

---

Step 0: Prerequisites

Windows PC (Windows 10 or 11 recommended)

Python 3.8+ installed
If not installed: Download Python

Make sure **"Add Python to PATH"** is checked during installation

Optional: **VS Code** (or PyCharm) for IDE

Command Prompt or PowerShell for creating virtual environments

---

Step 1: Create a Project Folder

Open File Explorer and create a folder for your project, e.g.:

D:\TrustGuardProject

Open **Command Prompt** or **PowerShell**, and navigate to this folder:

cd D:\TrustGuardProject

Step 2: Create a Python Virtual Environment

A virtual environment keeps dependencies isolated.
Create a virtual environment named 'env'

python -m venv env

Activate the environment
On PowerShell:

.\env\Scripts\Activate.ps1

On Command Prompt:

.\env\Scripts\activate.bat

You should now see (env) at the start of your terminal prompt.

Step 3: Upgrade pip (optional but recommended)

python -m pip install --upgrade pip

Step 4: Install Required Packages

Install Pandas, tqdm, and TrustGuard:

pip install pandas tqdm trustguard

Optional for visualization:

pip install matplotlib

Step 5: Create Your Python Script

Open your project folder in VS Code (or any editor)

Create a new file: trustguard_validation.py

Start your script with a descriptive header:

```
# ---------------------------------------
# TrustGuard Validation Script
# Marks each row as Approved or Rejected
# Uses Pandas + TrustGuard (GenericResponse)
# ---------------------------------------
```

Step 6: Load Dataset

```
import pandas as pd

# Load CSV dataset
df = pd.read_csv("Sentiment140_clean.csv")
```

```
print("Dataset loaded! Total rows:", len(df))

# Optional: inspect first rows
print(df.head())
```

Step 7: Initialize TrustGuard

```
from trustguard import TrustGuard
from trustguard.schemas import GenericResponse

# Initialize TrustGuard with GenericResponse schema
guard = TrustGuard(schema_class=GenericResponse)
```

Step 8: Validate Each Row

```
import json
from tqdm import tqdm

approval_list = []

for _, row in tqdm(df.iterrows(), total=len(df)):
    text = str(row["Text"])

    # Build JSON for TrustGuard
    json_payload = {
        "content": text,
        "sentiment": "neutral",
        "tone": "neutral",
        "is_helpful": True
    }
    json_text = json.dumps(json_payload)

    # Validate
    result = guard.validate(json_text)

    if result.is_approved:
        approval_list.append("Approved")
    else:
        approval_list.append("Rejected")

# Add column to DataFrame
df["sent"] = approval_list
```

Step 9: Inspect Rejected Rows

```
rejected_rows = df[df["sent"] == "Rejected"]
print(rejected_rows[["Text"]].head(10))
print("Total rejected rows:", len(rejected_rows))
```

Step 10: Summary Statistics Using TrustGuard

```
# Get stats directly from TrustGuard
stats = guard.get_stats()
print(stats)

# Optional: calculate percentages
total = stats['total_validations']
approved_pct = stats['approved'] / total * 100
rejected_pct = stats['rejected'] / total * 100

print(f"Approved: {approved_pct:.2f}%")
print(f"Rejected: {rejected_pct:.2f}%")
```

Expected output:

```
{'total_validations': 1600000, 'approved': 1487997, 'rejected': 112003, 'judge_checks': 0}
Approved: 92.99%
Rejected: 7.01%
```

Step 11: Save the Resulting Dataset

```
df.to_csv("Sentiment140_with_sent_status.csv", index=False, encoding="utf-8")
print("      Dataset saved with Approved/Rejected status!")
```

Step 12: Optional Enhancements

Visualize results:

```
import matplotlib.pyplot as plt

df['sent'].value_counts().plot(kind='bar', color=['green','red'], title="Approval Status")
plt.show()
```

Inspect common words in rejected rows:

```
from collections import Counter
```

Outcome of this Study Guide

Virtual environment with all dependencies isolated

Dataset validated with TrustGuard

sent column shows **Approved/Rejected**

Rejected rows can be inspected for quality issues

Summary stats and percentages available

Dataset saved for further analysis

The complete code

```
# ---------------------------------------
# TrustGuard Validation Script
# Marks each row as Approved or Rejected
# Uses Pandas + TrustGuard (GenericResponse)
# ---------------------------------------

import pandas as pd
import json
from trustguard import TrustGuard
from trustguard.schemas import GenericResponse
from tqdm import tqdm
import matplotlib.pyplot as plt

# ----------------------------
# Step 1: Load Dataset
# ----------------------------
df = pd.read_csv("Sentiment140_clean.csv")
print("Dataset loaded! Total rows:", len(df))
```

```
print(df.head())

# ----------------------------
# Step 2: Initialize TrustGuard
# ----------------------------
guard = TrustGuard(schema_class=GenericResponse)

# ----------------------------
# Step 3: Validate each row
# ----------------------------
approval_list = []

for _, row in tqdm(df.iterrows(), total=len(df)):
    text = str(row["Text"])

    # Build JSON payload for TrustGuard
    json_payload = {
        "content": text,
        "sentiment": "neutral",
        "tone": "neutral",
        "is_helpful": True
    }
    json_text = json.dumps(json_payload)

    # Validate
    result = guard.validate(json_text)

    if result.is_approved:
        approval_list.append("Approved")
    else:
        approval_list.append("Rejected")

# Add approval status column
df["sent"] = approval_list

# ----------------------------
# Step 4: Inspect Rejected Rows
# ----------------------------


rejected_rows = df[df["sent"] == "Rejected"]
print("\nFirst 10 rejected rows:")
print(rejected_rows[["Text"]].head(10))
print("\nTotal rejected rows:", len(rejected_rows))
```

```
# ----------------------------
# Step 5: TrustGuard Stats
# ----------------------------
stats = guard.get_stats()
print("\nTrustGuard Statistics:")
print(stats)

# Percentages
total = stats['total_validations']
approved_pct = stats['approved'] / total * 100
rejected_pct = stats['rejected'] / total * 100

print(f"\nApproved: {approved_pct:.2f}%")
print(f"Rejected: {rejected_pct:.2f}%")

# ----------------------------
# Step 6: Optional - Visualize Approved vs Rejected
# ----------------------------
df['sent'].value_counts().plot(kind='bar', color=['green','red'], title="Approval Status")
plt.show()

# ----------------------------
# Step 7: Save Updated Dataset
# ----------------------------
df.to_csv("Sentiment140_with_sent_status.csv", index=False, encoding="utf-8")
print("\n      Dataset saved as 'Sentiment140_with_sent_status.csv'")
```